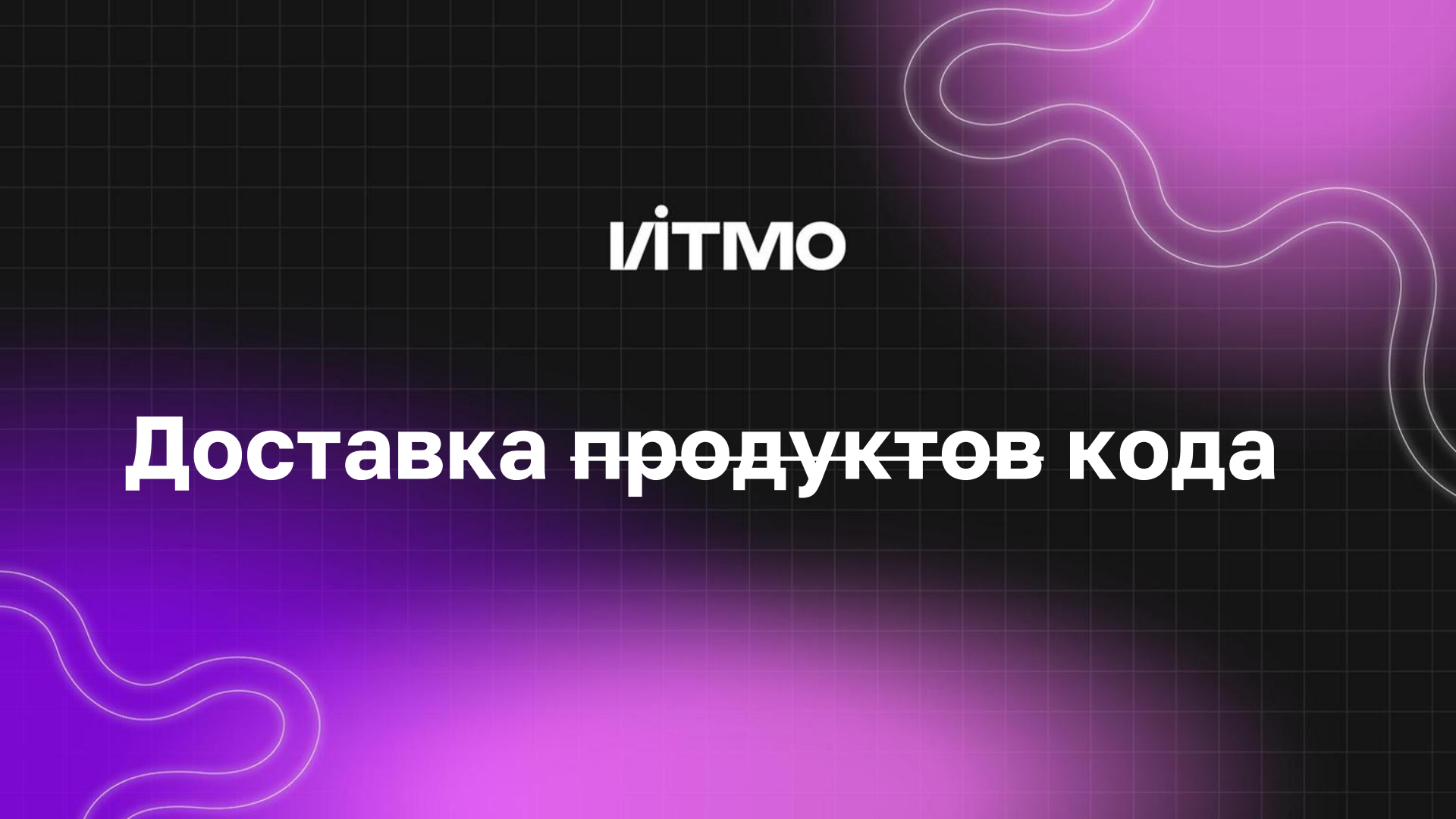


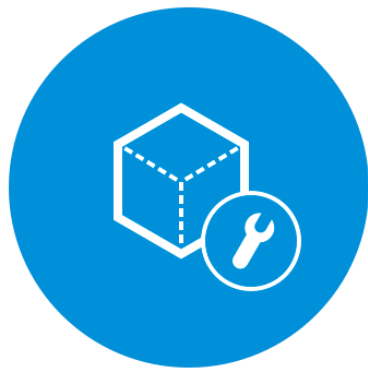


ІТМО

Доставка продуктів кода

Как код от разработчика попадает на сервер?

ІІТМО



Development



Testing/QA



Production



Как было раньше



Минусы подхода:



- Простой ПО («Сайт закрыт на техобслуживание, зайдите завтра»)
- Долго выкатывается любое изменение
- Нужно много специалистов для выкатки (разработчики, тестировщики, сисадмины)
- Высок риск ошибки при настройке ПО
- Много рутины при выкатке

Появился CI/CD



Continuous Integration



Непрерывная интеграция — процесс постоянной разработки ПО с интеграцией в основную ветвь. Автоматически собирает софт, тестирует его и оповещает, если что-то идёт не так.

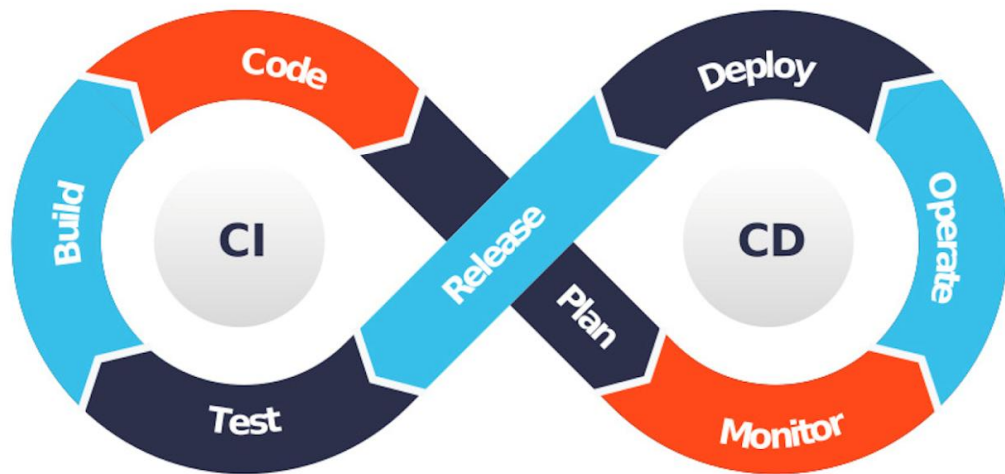
Continuous Delivery



Непрерывная доставка — процесс постоянной доставки ПО до потребителя. Обеспечивает разработку проекта небольшими частями и гарантирует, что он может быть отдан в релиз в любое время без дополнительных ручных проверок.

Плюсы CI/CD

ІТМО



- Не допустит ошибки до прода
- Улучшение качества кода
- Повышает скорость разработки
- Решает проблему “а у меня локально работает”
- Гибко и масштабируемо применяет настройки

Непрерывная интеграция (Continuous Integration/CI)



Непрерывная интеграция – практика разработки ПО, при которой ПО **собирается и тестируется** каждый раз, когда разработчик пушит код приложения и происходит несколько раз в день

Непрерывная интеграция: TEST - BUILD

Непрерывная доставка (Continuous Delivery)

Непрерывная доставка – подход к разработке ПО, при котором возможности CI, автоматизированного тестирования и автоматизированного развертывания позволяют ПО быть разработанным и развернутым быстро и надежно с минимальным человеческим вмешательством. Развертывание в production среде должно быть заранее обговорено и вызвано вручную.

Непрерывная интеграция:
TEST – BUILD – DEPLOY ВРУЧНУЮ

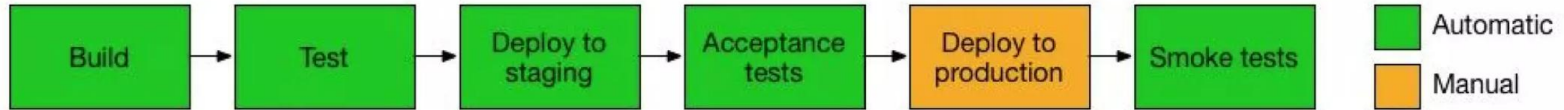
Непрерывное развертывание (Continuous Deployment/CD)



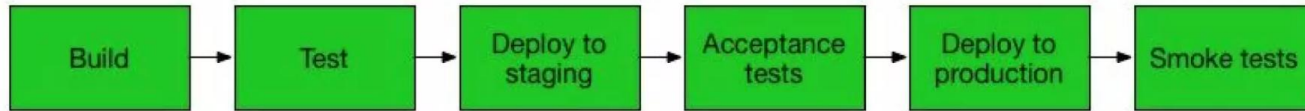
Непрерывное развертывание – подход к разработке ПО, при котором каждое изменение в коде идет через весь пайплайн с автоматическим развертыванием в среду production, что вызывает больше количество деплоев в среде production каждый день, т. е. процесс – полностью автоматизированный Continuous Delivery.

Непрерывная интеграция:
TEST – BUILD – DEPLOY АВТОМАТОМ

Continuous Delivery vs Continuous Deployment



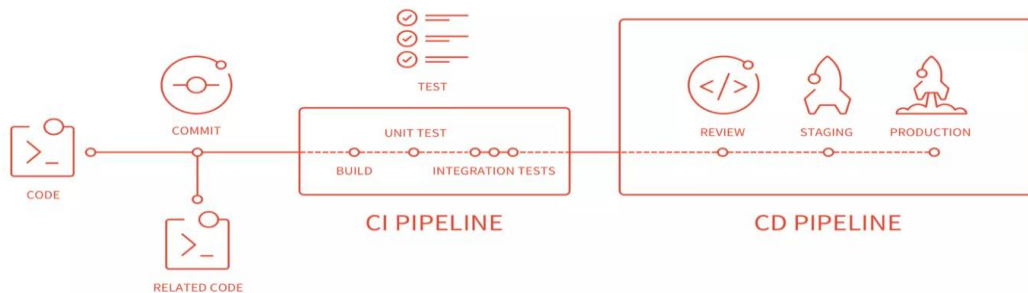
Continuous Delivery



Continuous Deployment

CI/CD Pipeline (CI/CD конвейер)

CI/CD PIPELINE



<https://docs.gitlab.com/ce/ci/>

Пример простого CI/CD



test



test-job



dev



build-dev



deploy-dev



prod



build-prod



deploy-prod



monitoring



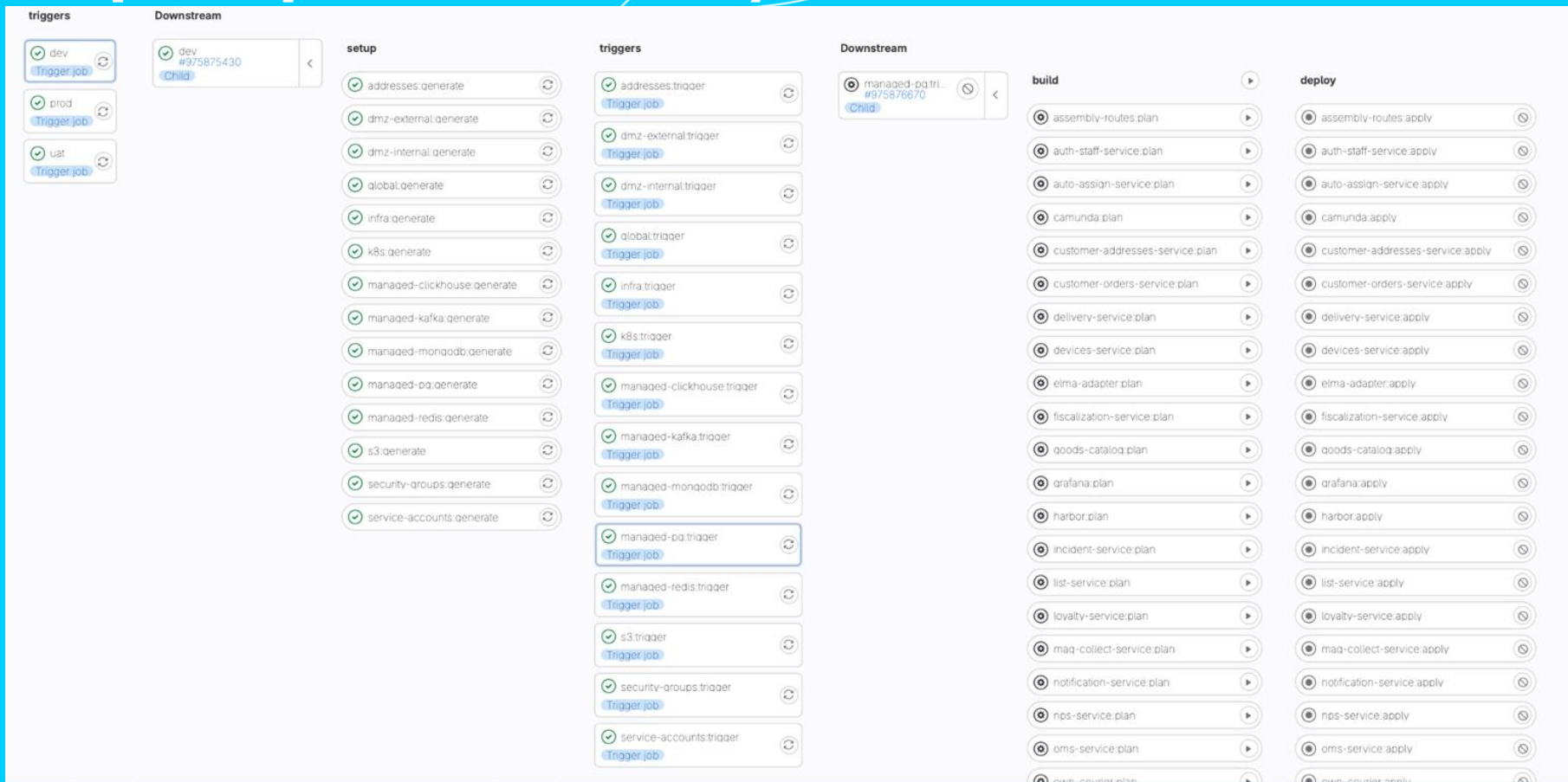
deploy-monitoring-dev



deploy-monitoring-prod

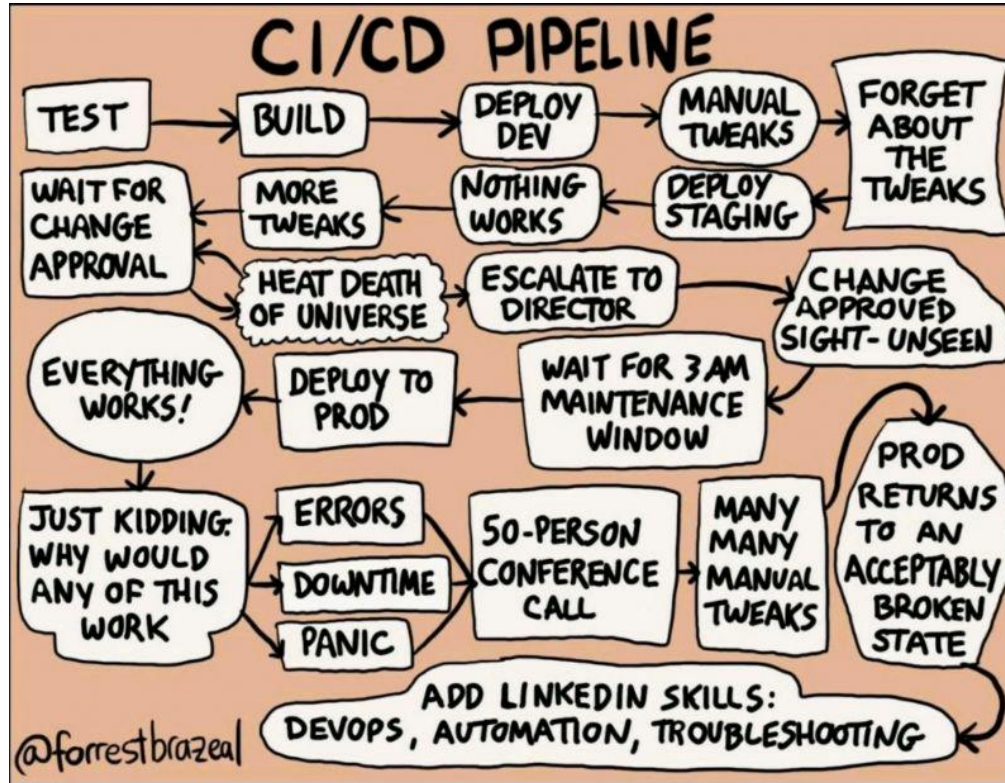


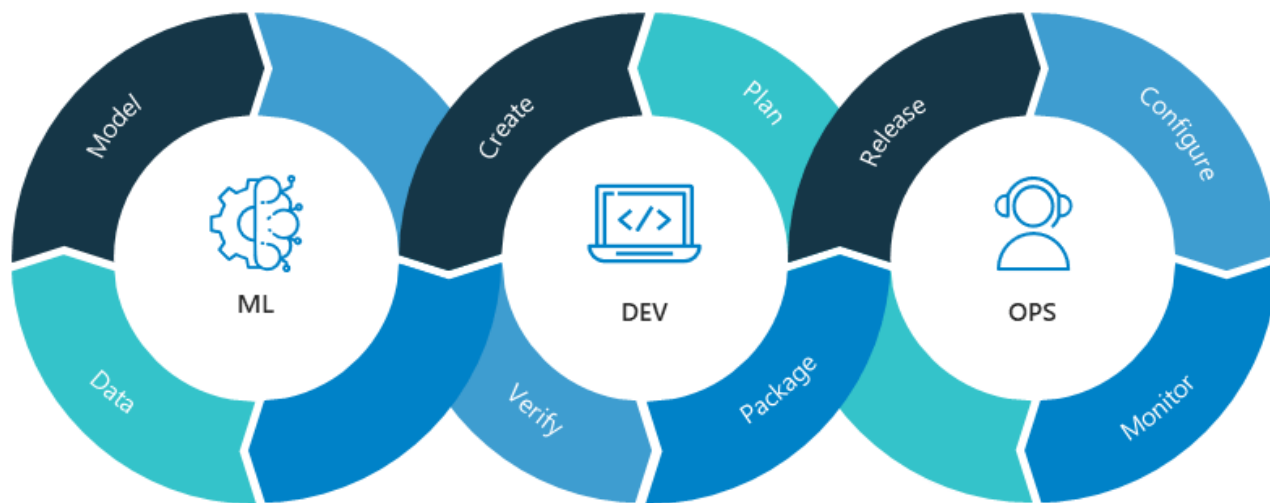
Пример сложного CI/CD



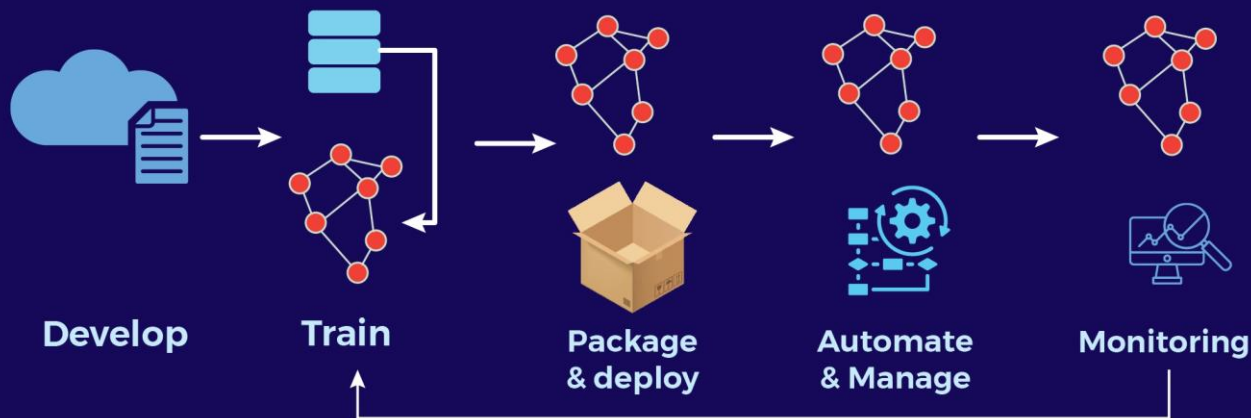
Ha-ha, classic

VITMO

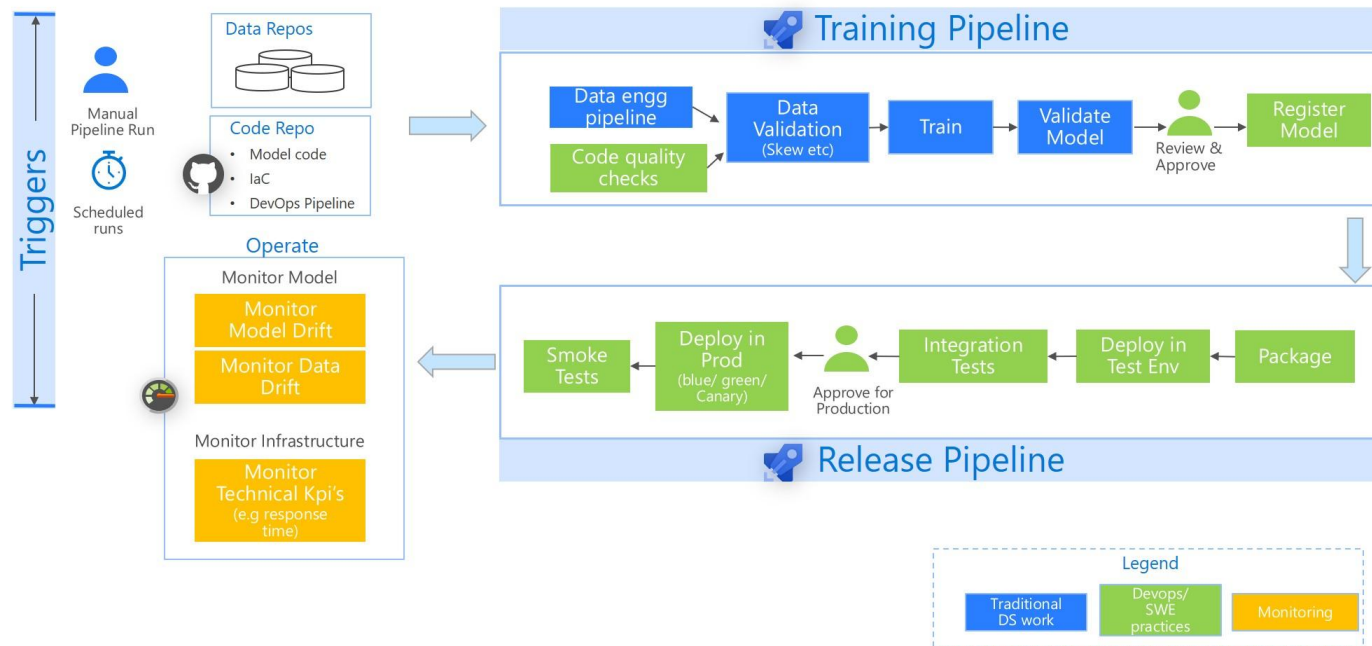




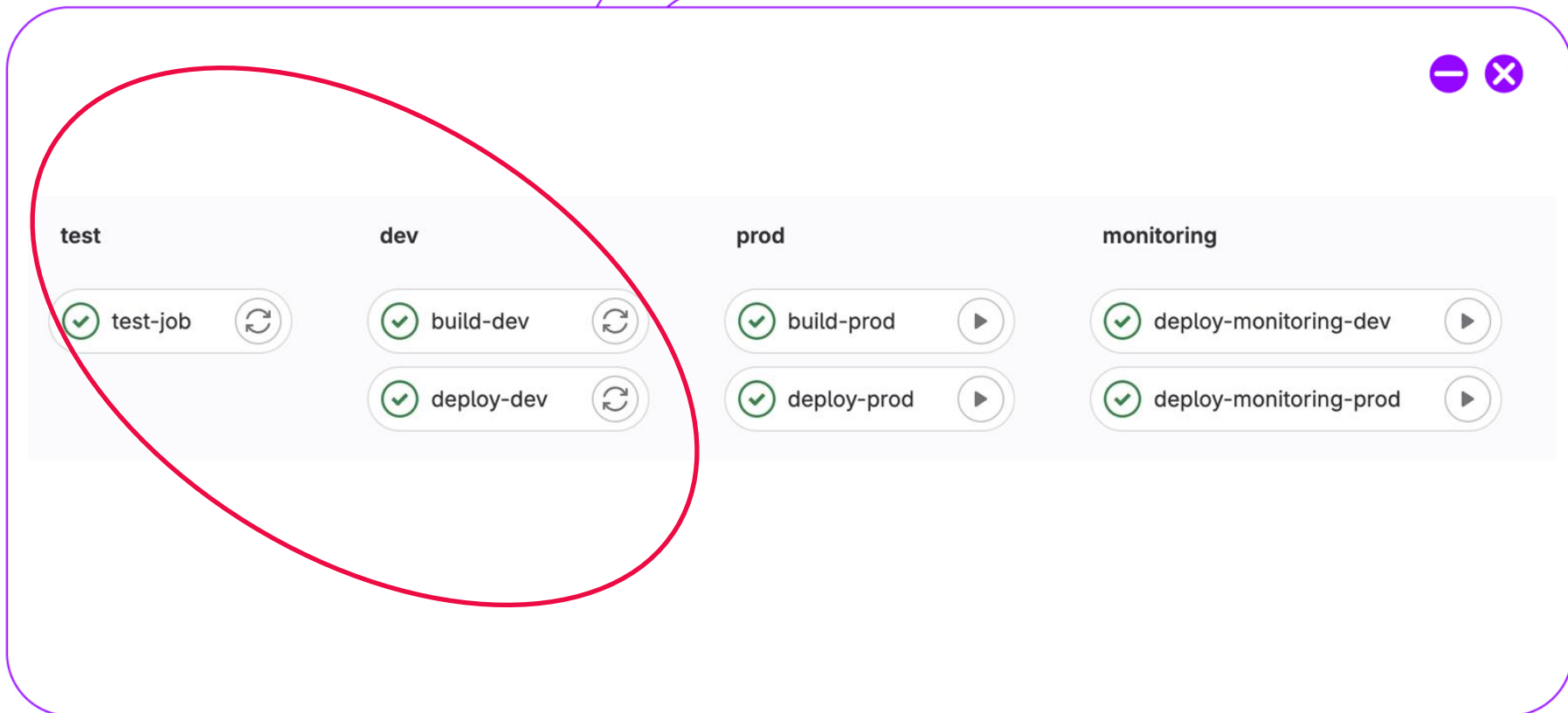
MLOps Lifecycle



MLOps Flow



Пример простого CI/CD



Плохие практики



- Пушим всё сразу в мейн
- Всё пишет один человек :)
- Непонятные подписи у коммитов
- Пушим всё в одну ветку test, в мейне пусто
- Что-нибудь ещё?



Хорошие практики



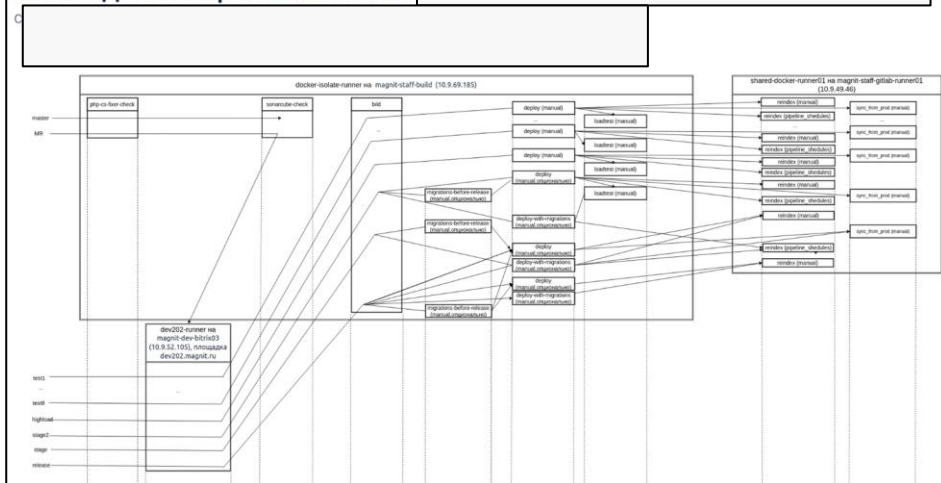
- **Коммитьте часто**, маленькими порциями
- **Понятные commit messages** (conventional commits)
- **Не коммитьте секреты** (.gitignore, git-secrets)
- **Используйте .gitignore**
- **Делайте rebase** перед merge (чистая история)
- **Code review** через Merge Requests

Стратегия ветвления

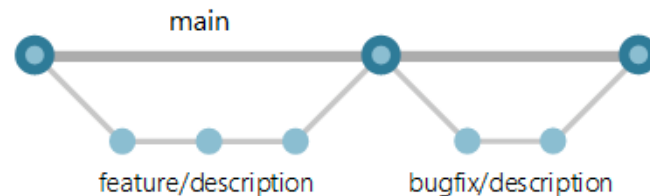
Стратегия ветвления - организация разработки проекта ПО с помощью системы управления версиями (в данном случае *git*), определяющая правила ветвления, интеграции и доставки кода



Схема деплоя проекта backend



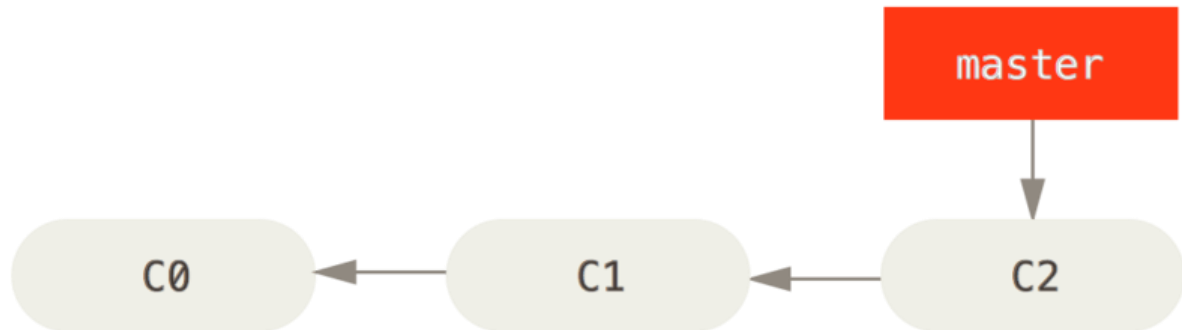
Работа с кодом может быть такой



А может быть такой

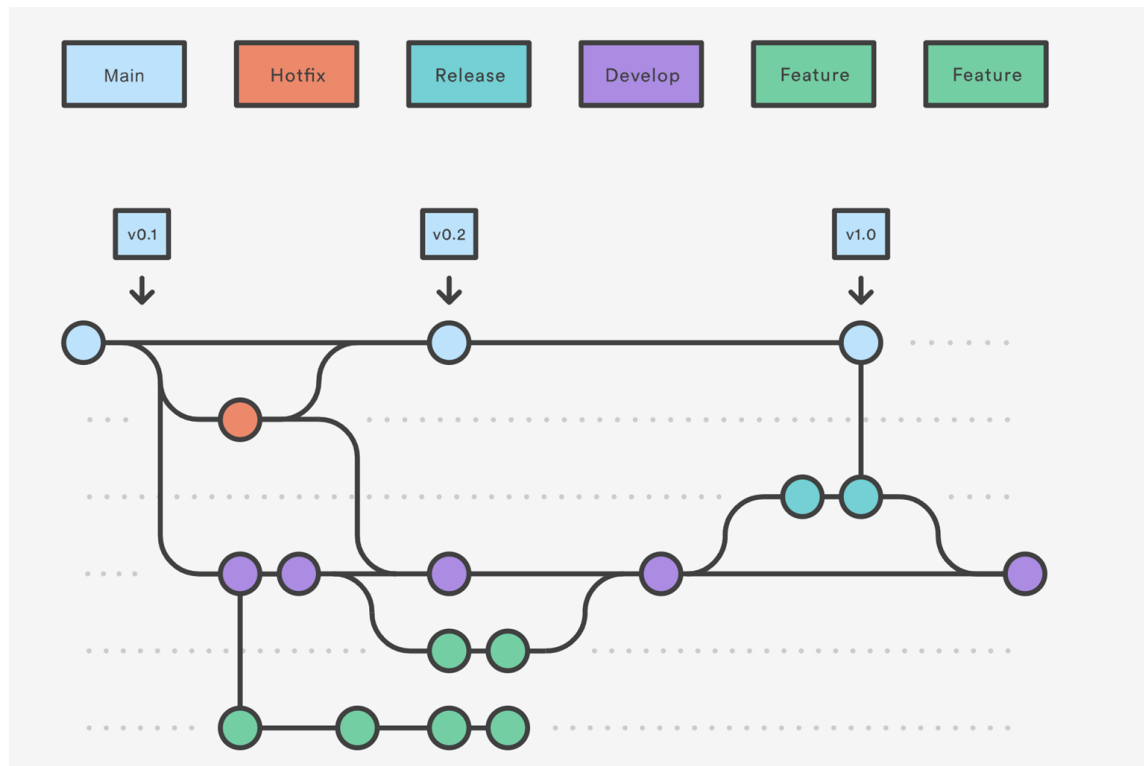
ONE BRANCH FLOW

- Вся разработка происходит в одной ветке
- Все изменения сохраняются в одну ветку
- Разворачивание кода происходит из той же ветки



GIT FLOW

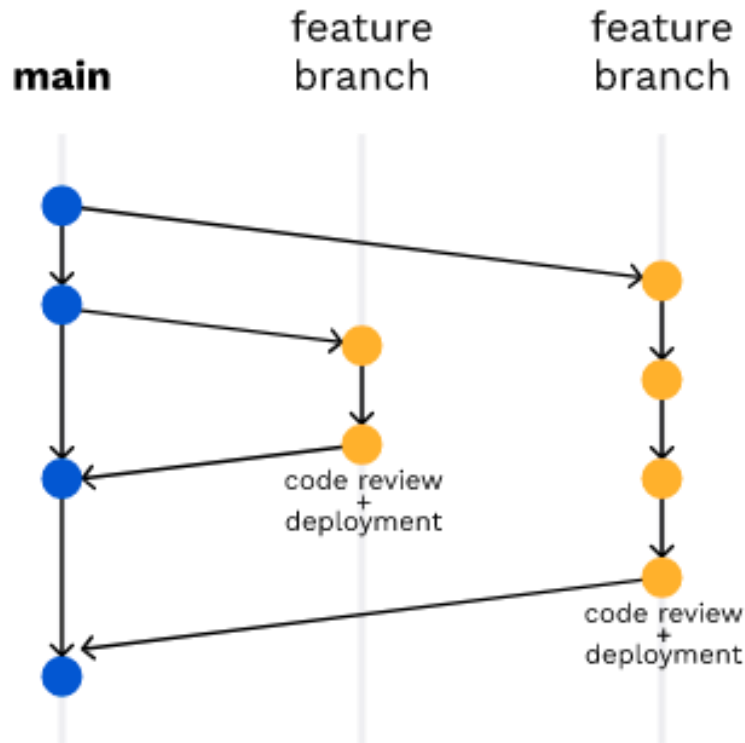
- Ветка для разработки от ветки main
- Для каждого небольшого изменения feature ветка от ветки develop
- Когда изменений достаточно для полноценного релиза – ветка release
- Небольшие изменения можно вливать сразу в main



Изображение из документации git-scm.com

GITHub FLOW

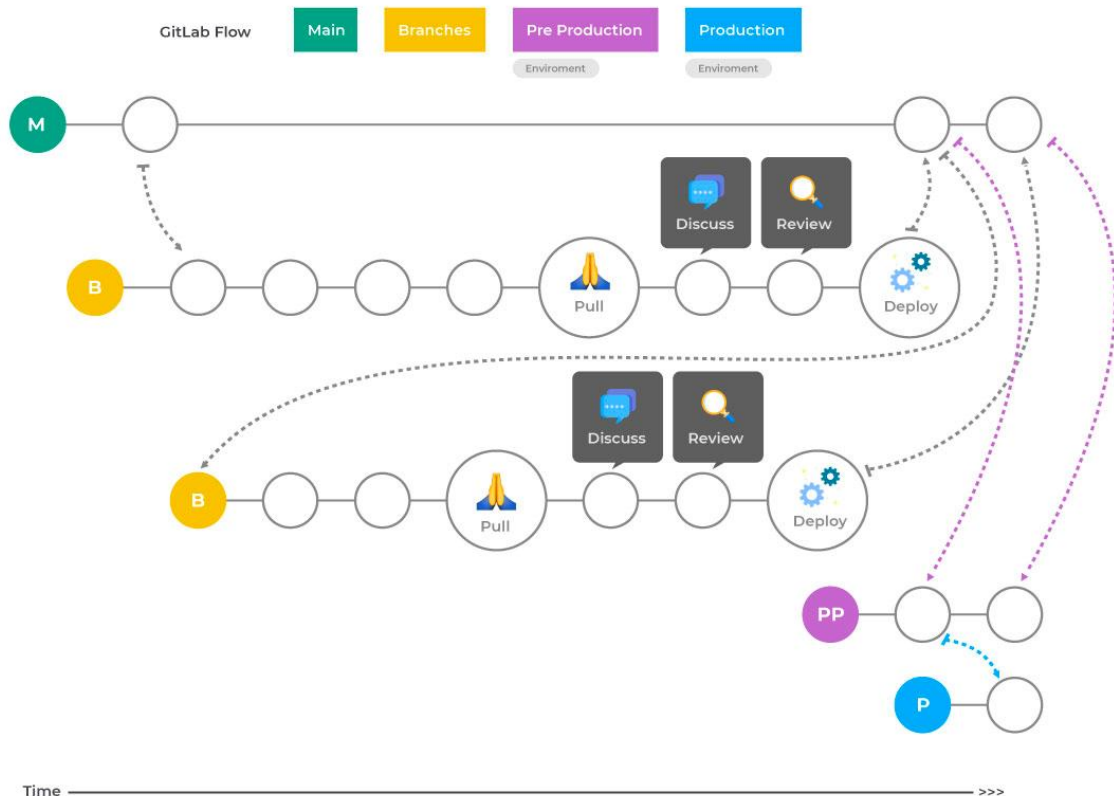
- Для каждого небольшого изменения feature ветка от ветки main
- Когда изменение готово - они вносятся в main
- Новые изменения в main необходимо добавлять в другие feature ветки



Изображение из документации
github.com

GITLAB FLOW

- Создание feature веток аналогично GitHub Flow
- Ветка под Pre-Production для тестирования окружения
- Ветка под Production, из которой код разворачивается в релиз

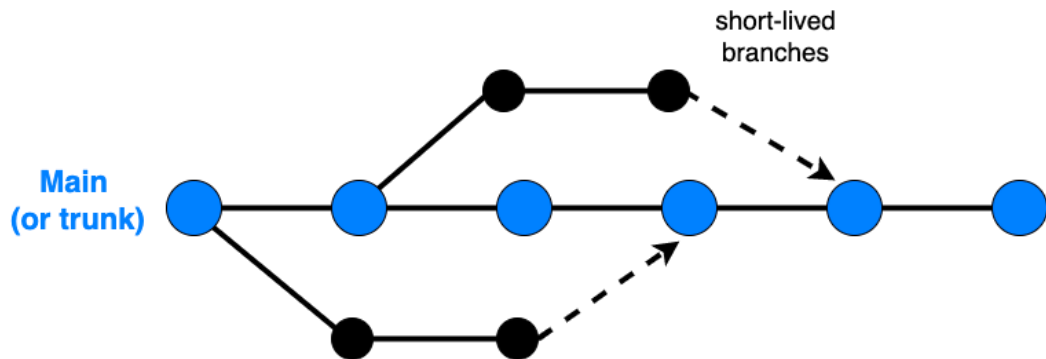


Изображение из документации
gitlab.com

TRUNK BASED FLOW

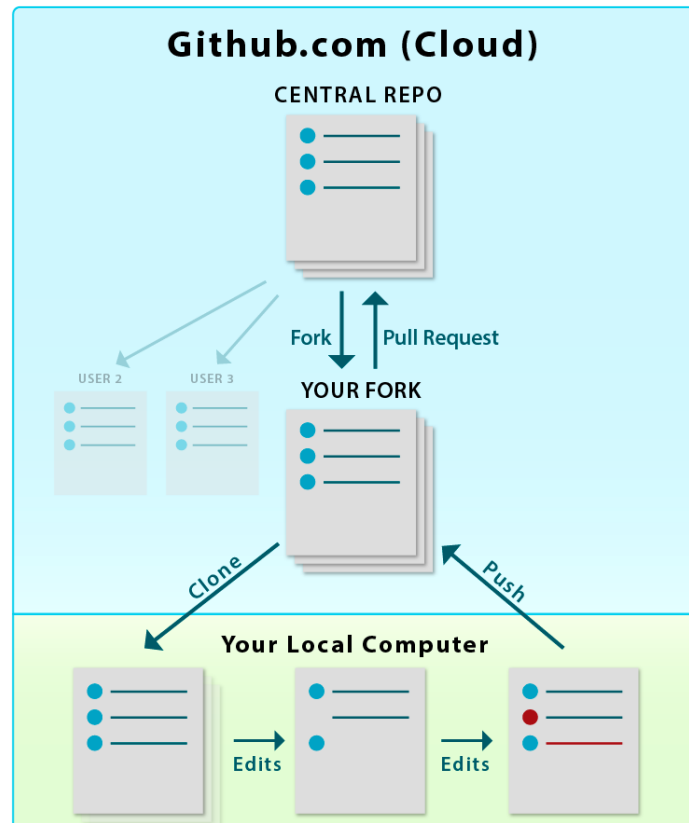
- Все изменения сразу же вносятся в main – feature ветка не существует более одного дня
- Для неготовых изменений в main используются feature флаги - код выключен, пока не доработан
- Быстрая проверка кода, так как нет больших изменений

Trunk-based development



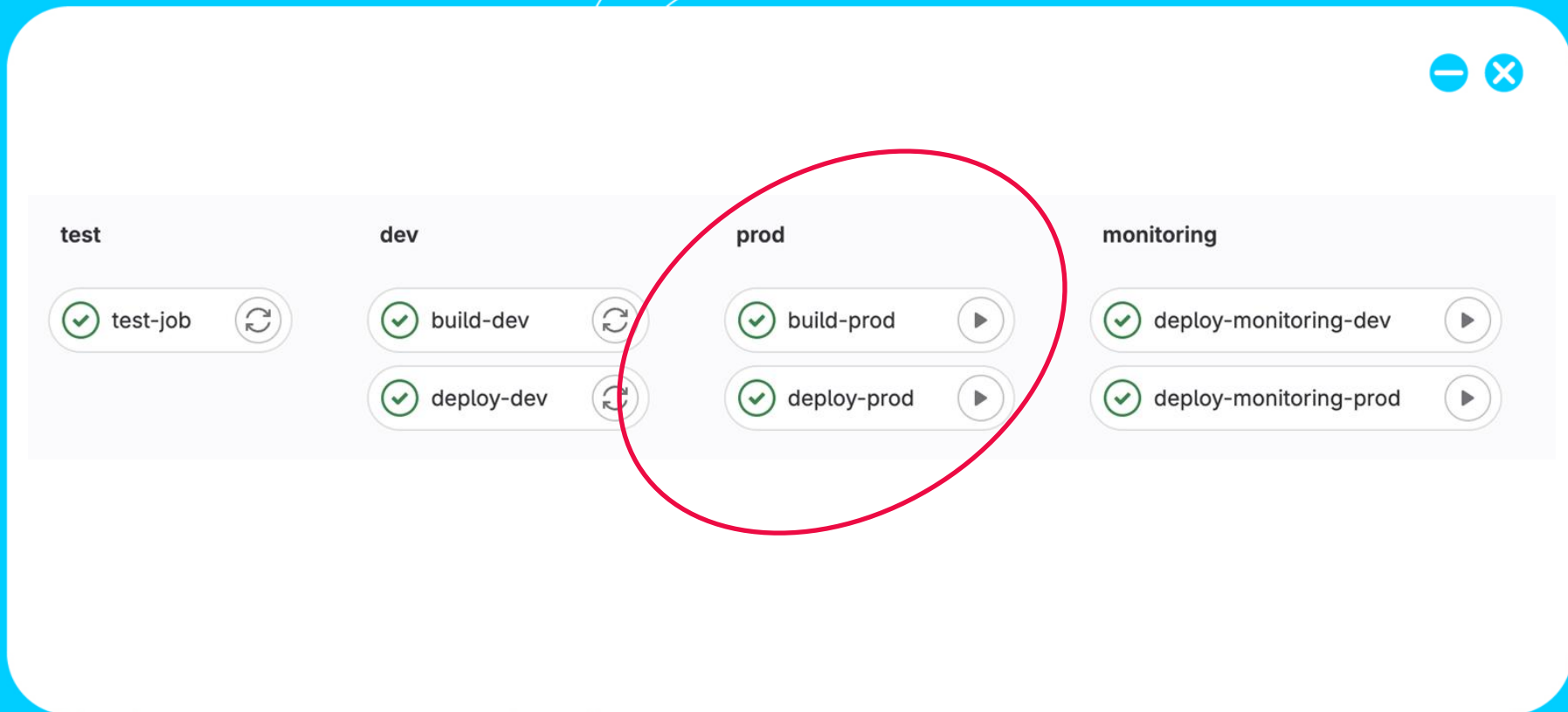
FORKING FLOW

- Наличие основного репозитория с кодом
- Все изменения вносятся в fork репозитория каждого пользователя
- Если изменения качественные, то они принимаются в основном репозитории
- Часто используется в разработке с открытым исходным кодом



Изображение из документации
github.com

Пример простого CI/CD



Простой деплой



Все узлы обновляются
одновременно новой версией



Плюсы:

- Просто
- Быстро
- Дешево

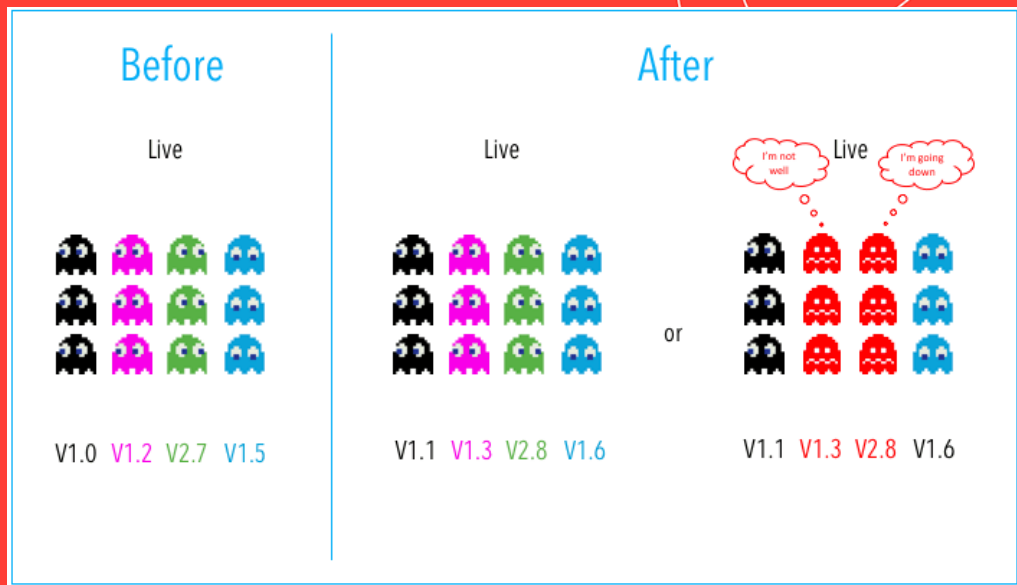
Минусы:

- Самая рискованная стратегия
- Не защищена от сбоев
- Сложный откат
- Не best practice

Когда использовать:

Некритичные сервисы, lower
окружения, off-hours

Multi-Service



Все узлы обновляются
несколькими сервисами
одновременно



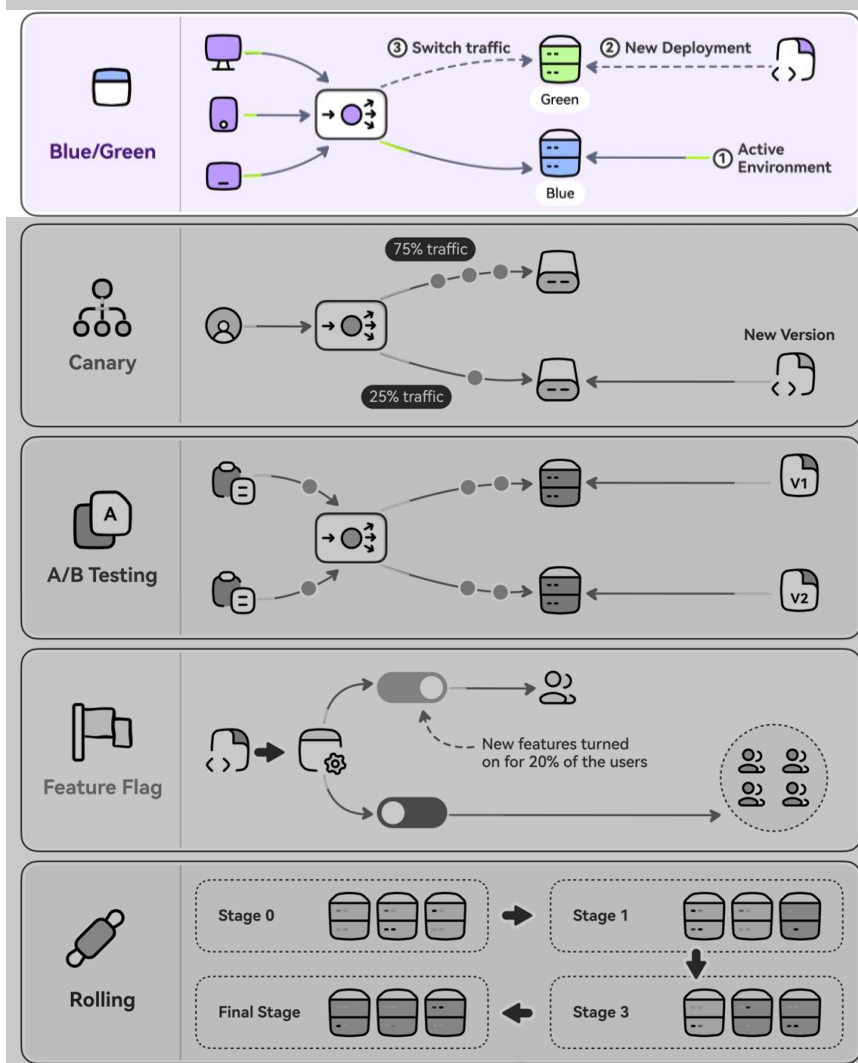
Плюсы:

- Просто
- Быстро
- Дешево
- Меньше рисков чем Basic

Минусы:

- Медленный откат
- Не защищена от сбоев
- Сложно управлять зависимостями
- Сложно тестировать все зависимости

Когда использовать: Сервисы с зависимостями, off-hours деплой



Две идентичные среды (Blue=staging, Green=production), переключение трафика между ними

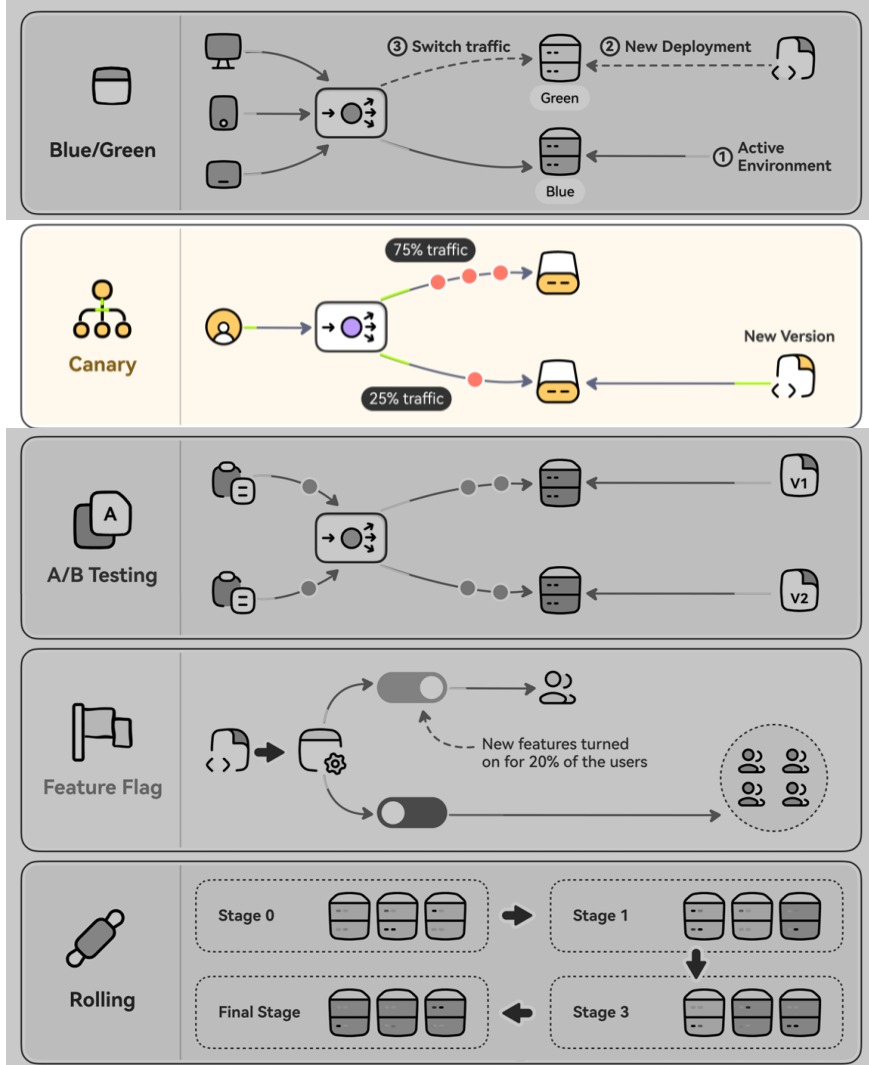
Плюсы:

- Простое и быстрое
- Мгновенный откат (переключить трафик обратно)
- Хорошо изучено
- Тестирование перед переключением

Минусы:

- Дорого (нужны 2 полных окружения)
- In-flight транзакции могут потеряться при переключении
- QA может не поймать все проблемы
- Проблема влияет сразу на всех пользователей

Когда использовать: Критичные сервисы с бюджетом на инфраструктуру



Постепенный выкат на процент пользователей (2% → 25% → 75% → 100%)

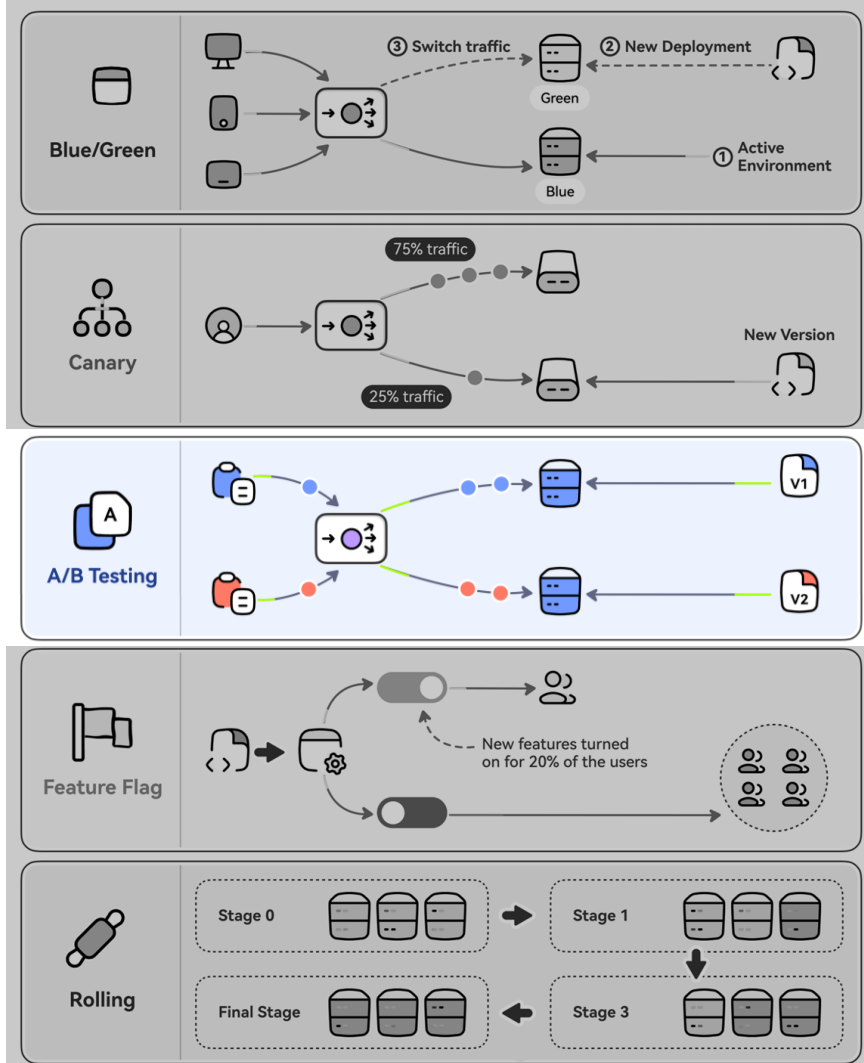
Плюсы:

- Наименьший риск из всех стратегий
- Тестирование на реальных пользователях
- Дешевле чем Blue-Green
- Быстрый и безопасный откат
- Сравнение версий side-by-side

Минусы:

- Сложная автоматизация/скриптинг
- Требуется мониторинг и метрики
- Тестирование в production (риск)
- Ручная проверка занимает время

Когда использовать: Production сервисы, нужен контроль над рисками



Разные версии работают одновременно как "эксперименты", трафик распределяется по правилам

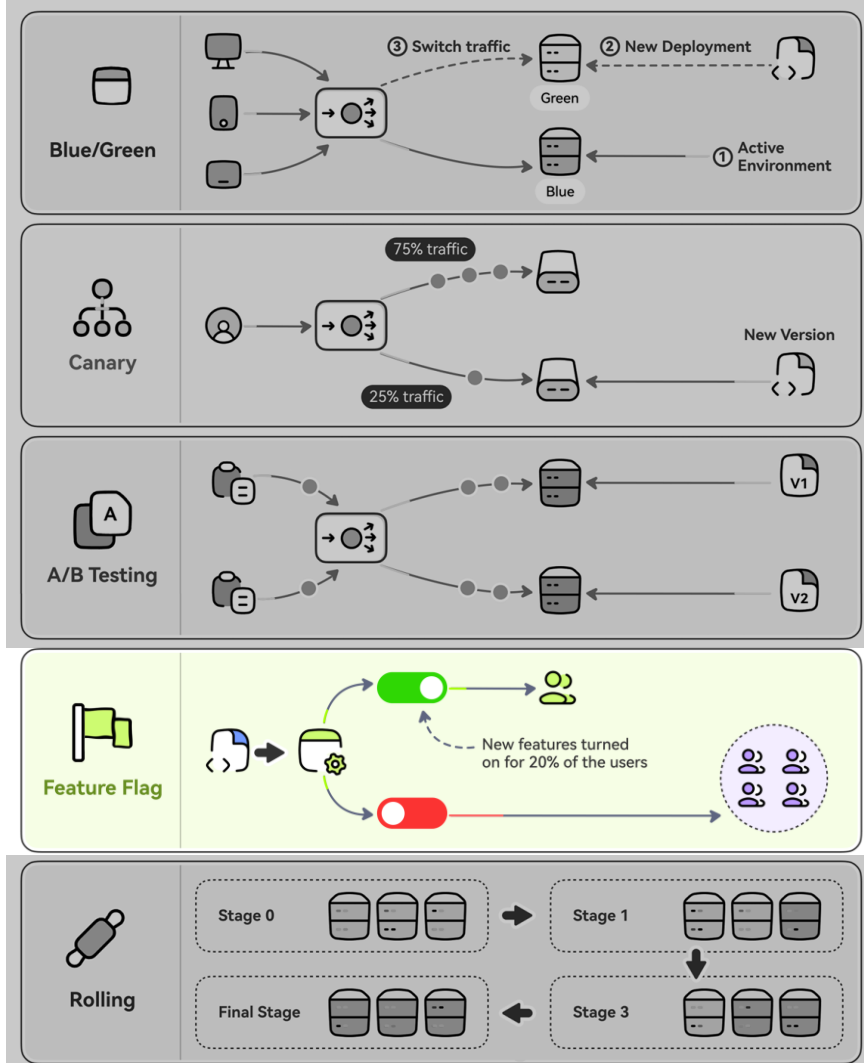
Плюсы:

- Стандартный метод тестирования фич
- Простой и дешевый
- Много готовых инструментов
- Данные для принятия решений

Минусы:

- Эксперименты могут сломать UX
- Сложная автоматизация
- Не про deployment, про экспериментирование
- Требует feature flags или A/B инструментов

Когда использовать: Тестирование гипотез, выбор между вариантами фич



Код новой фичи деплоится выключенным, включается через конфиг/переменную без передеплойа

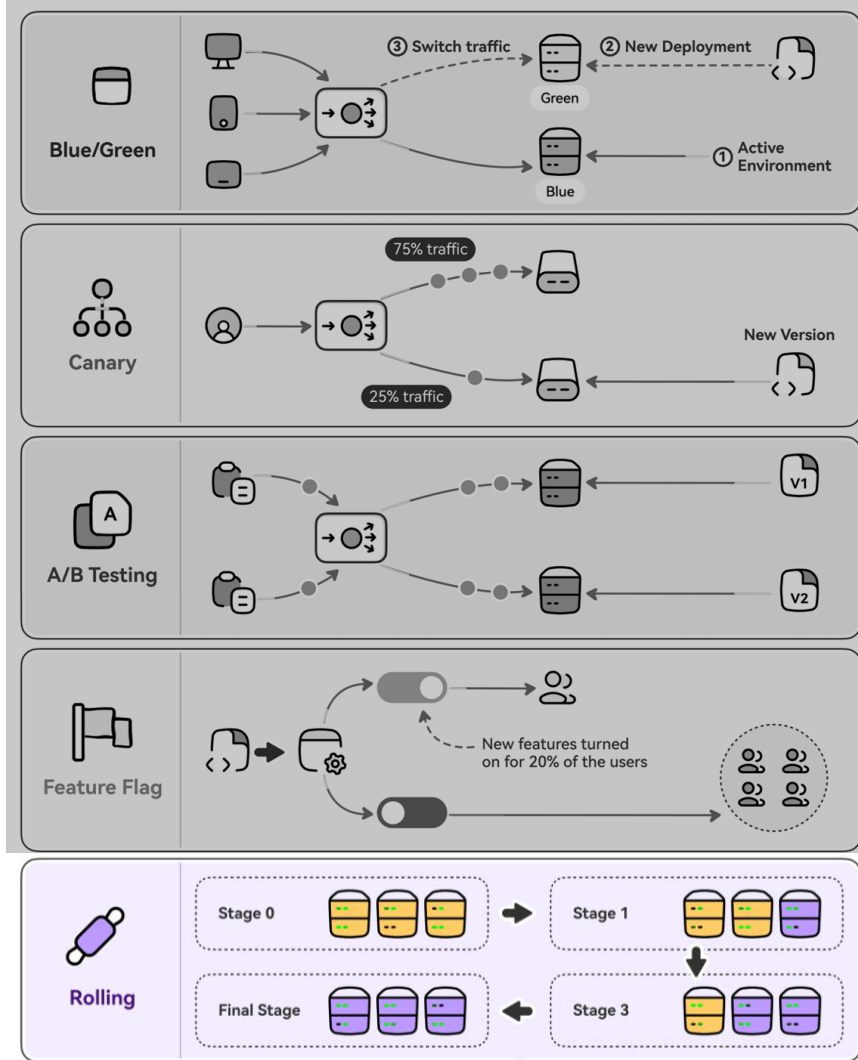
Плюсы:

- Отделяем deployment от release
- Деплой без риска (фича выключена)
- Мгновенный откат (выключил флаг)
- Постепенный rollout (% пользователей)
- A/B тестирование
- Trunk-based development (всё в main)

Минусы:

- Технический долг (старый код висит)
- Усложняет код (if/else ветвления)
- Нужно чистить старые флаги
- Требует инфраструктуру управления флагами
- Сложность тестирования (все комбинации)

Когда использовать: Continuous Deployment, большие фичи, эксперименты, постепенный rollout



Узлы обновляются постепенно партиями (batches)

Плюсы:

- Относительно простой откат
- Меньше рисков
- Простая реализация

Минусы:

- Сервисы должны поддерживать старую и новую версии
- Медленное (проверка на каждом шаге)
- Может быть несогласованность версий

Когда использовать: Stateless приложения, можно жить с временной несогласованностью

Подведём итоги

- Процесс доставки кода на сервер
- CI/CD
- Аккуратность при работе с гитом
- Стратегии ветвления и деплоя

